Submission

Current Solution

The corridor follows an existing cleared track, making it practical to implement and maintain. There is currently a single east-west firebreak corridor along row 10, with 17 cells in the centre and 2 open on each side. Even though it intercepts some fire from the southernmost ignition row, there are weaknesses. Wind from different angles can push fire through gaps or around the corridor. The entire budget is on a single fixed barrier, leaving no budget for reactive cells once wind conditions are revealed. The settlement and wetland cost different damage weights (10 and 5 respectively), and the corridor doesn't represent this; it will treat both targets the same regardless.

Proposal

With the results and the authority's preference for permanent plans, I propose clearing 17 cells permanently before fire season placing a barrier along row 16. This plan has the best risk profile (mean=1.73, $\text{CVaR}_{0.9}$=9.76 confirmed in Step 4) in comparison to the current corridor (mean=3.76, CVaR=24.32) which uses the same number of cells at row 10. It captures fire strength more realistically than the binary propagation model, where damage decreases smoothly as budget increases, instead of threshold behaviour. A fully permanent budget (e.g. B=17) is better than a reactive split (B=13, R=4) as the reactive MILP returned an optimal objective value of 70, which is equal to the permanent plan's no-wind damage, indicating that R=4 does not reduce predetermined reachability damage as significantly as expected, given permanent firebreak placement.

Scaffolding

Step 0:
Create a network to model this as a max-flow problem.

```r
# Load data - simulator, landscape, target
source("fire-simulator.r")
landscape <- as.matrix(read.csv("landscape.csv", header = FALSE))
targets <- read.csv("targets.csv")

# Edge capacities interpret spread probabilities, so find edges with no wind
edges <- spread_probabilities(landscape, wind_speed = 0, wind_dir = 0)

# Using rAMPL to pass data from R directly to AMPL
library(rAMPL)
environment <- new(Environment, "/Applications/AMPL")
ampl <- new(AMPL, environment)

# Must convert edges (row and column pairs) to named nodes
```

```r
edges$from_node <- paste0("r", edges$from_row, "_c", edges$from_col)
edges$to_node <- paste0("r", edges$to_row, "_c", edges$to_col)

# Needs a full list of unique nodes to define NODE set
total_nodes <- unique(c(edges$from_node, edges$to_node))
length(total_nodes) # Should be 439, 441 - 2 which are bare with no classification

# Source nodes on southernmost row (21)
source_nodes <- edges$from_node[edges$from_row == 21]
source_nodes <- unique(source_nodes)
# 21 source nodes; once per cell in row 21

# Target nodes
settlement <- targets[targets$weight == 10, ]
wetland <- targets[targets$weight == 5, ]
settlement_node <- paste0("r", settlement$row, "_c", settlement$col)
wetland_node <- paste0("r", wetland$row, "_c", wetland$col)
```

Step 1: Network bottleneck model

Let V be a set of nodes, A be the set of arcs, s the super source and t the super target. The max-flow problem is:

$$\max x_{ts}$$
$$\text{subject to } \Sigma_{(v,j)\in A} x_{vj} - \Sigma_{(i,v)\in A} x_{iv} = 0, \forall v \in V$$
$$0 \leq x_{ij} \leq u_{ij}, \forall (i,j) \in A$$

where $c_{ij} = p_{burn}\,(i \to j)$ is the spread probability as edge capacity and x_{ts} is the maximised return arc flow.

Plain Text

```
# Max-flow problem in AMPL, maxflow.mod

# Sets
set NODES;
set ARCS within {NODES, NODES} # Within product set

# Parameters
param capacity {ARCS} >= 0;
param source symbolic in NODES;
param target symbolic in NODES;

# Variables
var flow {(i,j) in ARCS} >= 0;
var F >= 0;

```

```
# Objective: maximise flow through return arc
maximise TotalFlow:
    F;

# Flow conservation at every node
subject to Conservation {v in NODES}:
    sum {(v,j) in ARCS} flow[v,j]
     - sum {(i,v) in ARCS} flow [i,v]
     = if v = source then F
       else if v = target then -F
       else 0;

# Capacity constraints
subject to Capacity {(i,j) in ARCS}:
    flow[i,j] <= capacity[i,j];
```

R

```r
# Create a 'super' source and target node for max-flow problem
southernmost_source_edges <- data.frame(
    from_node = "s",
    to_node = source_nodes,
    p_burn = 1000000 # Large enough so will never become bottleneck
)

settlement_sink_edges <- data.frame(
    from_node = settlement_node,
    to_node = "t",
    p_burn = 1000000
)

# Create max-flow settlement network
settlement_network <- rbind(
    edges[, c("from_node", "to_node", "p_burn")],
    southernmost_source_edges,
    settlement_sink_edges,
    data.frame(from_node = "t", to_node = "s", p_burn = 1000000)
)

# 441 including super source and target
entire_settlement <- unique(c(settlement_network$from_node,
settlement_network$to_node))

# Repeat for wetland
wetland_sink_edges <- data.frame(
```

```
    from_node = wetland_node,
    to_node = "t",
    p_burn = 1000000
)

wetland_network <- rbind(
    edges[, c("from_node", "to_node", "p_burn")],
    southernmost_source_edges,
    wetland_sink_edges,
    data.frame(from_node = "t", to_node = "s", p_burn = 1000000)
)

entire_wetland <- unique(c(wetland_network$from_node,
wetland_network$to_node))
```

2. Solve max-flow problem and report shadow prices to identify bottleneck corridors.

```R
ampl$reset()
ampl$read("maxflow.mod")
ampl$setOption("solver", "gurobi")

# For settlement
# Load extended data into AMPL
ampl$getSet("NODES")$setValues(entire_settlement)
ampl$getParameter("source")$set("s")
ampl$getParameter("target")$set("t")

# Load arcs
ampl$getSet("ARCS")$setValues(settlement_network[, c("from_node",
"to_node")])

# Load capacities
ampl$getParameter("capacity")$setValues(settlement_network[, c("from_node",
"to_node", "p_burn")])

# Solve model
ampl$solve()
max_flow_settlement <- ampl$getValue("TotalFlow")
cat("Maximum fire flow to settlement is", max_flow_settlement, "\n")

# Store shadow prices of settlement
shadow_prices <- ampl$getConstraint("Capacity")$getValues()
settlement_shadow_prices <- shadow_prices[shadow_prices$Capacity.dual > 0, ]
```

```r

# Repeat for wetland
ampl$reset()
ampl$read("maxflow.mod")
ampl$setOption("solver", "gurobi")

ampl$getSet("NODES")$setValues(entire_wetland)
ampl$getParameter("source")$set("s")
ampl$getParameter("target")$set("t")

ampl$getSet("ARCS")$setValues(wetland_network[, c("from_node", "to_node")])

ampl$getParameter("capacity")$setValues(wetland_network[, c("from_node", "to_node", "p_burn")])

ampl$solve()
max_flow_wetland <- ampl$getValue("TotalFlow")
cat("Maximum fire flow to wetland is", max_flow_wetland, "\n")

shadow_prices <- ampl$getConstraint("Capacity")$getValues()
wetland_shadow_prices <- shadow_prices[shadow_prices$Capacity.dual > 0, ]
```

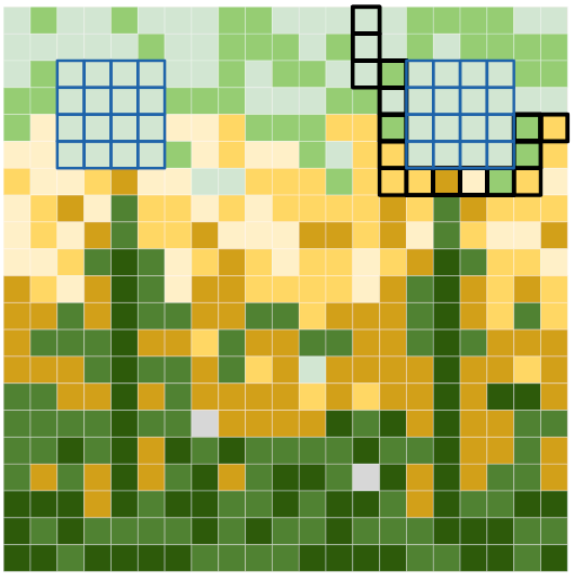
```r
# Look up each bottleneck node name in edges df, returning corresponding 'coordinates'
settlement_bottleneck_area <- data.frame(
    r = edges$from_row[match(settlement_shadow_prices$index0, edges$from_node)],
     c = edges$from_col[match(settlement_shadow_prices$index0, edges$from_node)]
)

wetland_bottleneck_area <- data.frame(
    r = edges$from_row[match(wetland_shadow_prices$index0, edges$from_node)],
     c = edges$from_col[match(wetland_shadow_prices$index0, edges$from_node)]
)
```

4.1.4

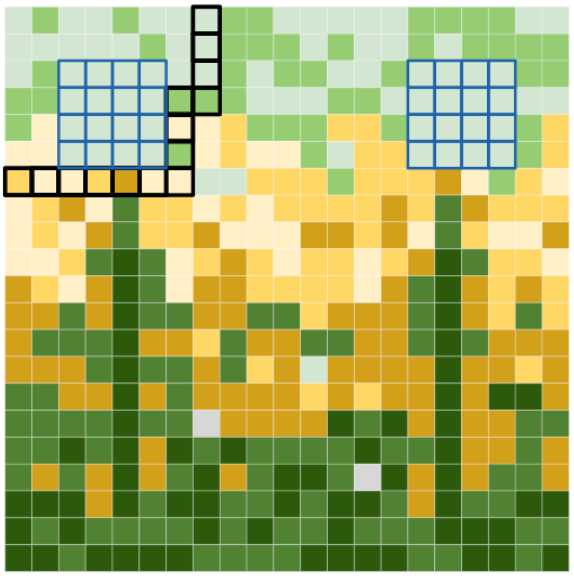
1) The current east-west corridor does not coincide with the bottlenecks as it acts like a wall centrally at row 10. The corridor does catch some southward fire but misses more concentrated areas in rows 1-7, which are the areas that the fire goes to reach the settlement and the wetland. The bottleneck matrices show that fire reaches the settlements through columns 14-21 and reaches the wetlands through columns 1-8.

2) The settlement has a higher maximum fire flow (4.06 > 3.795) and has more bottleneck edges than the wetland (38 > 34), so it is harder to protect.

Settlement bottleneck in rows 1-7, columns 14-21



Wetland bottleneck in rows 1-7, columns 1-8



3) The model ignores wind, which alters probabilistic spread and directs fire in specific directions. Ignition is fixed on the southernmost row, but normally, ignition location is randomised, and a fire may start closer to the settlement or the wetland, creating different fire flow patterns. Under west wind, fire pressure shifts towards columns 1-8, increasing flow toward the wetland and reducing flow toward the settlement. Min-cut corridors identified near columns 14-21 would no longer represent true bottlenecks, so the firebreak proposal from this step would be misplaced.
4) Since we've calculated max-flow, to find out which cells to clear, note that the non-zero shadow prices find the min-cut (by max-flow min-cut theory). But the bottleneck regions do not overlap, so clearing the cells will need two independent firebreak plans. A firebreak plan would be in two parts: to protect the settlement, clear the bottleneck cells around rows 1-7, columns 14-17 and rows 5-7, columns 19-21, and to protect the wetland, clear the bottleneck cells around rows 1-6, columns 6-8 and row 7, columns 1-7. If you want to protect both the settlement and the wetland, clear the cells from both sets. Since there is no budget, this should be feasible.

Step 2: Budget constrained fire-break optimisation

Let C be cells, I be ignition cells, E_τ be active edges ($p_{burn} > \tau$), with T targets, w_i damage weight and budget B.

Clear: $x_i \in \{0, 1\}$ and burn: $b_i \in \{0, 1\}$.

For Model A:

$$\min_{x,b} \Sigma_{i \in T} w_i b_i$$

$$\Sigma_{i \in C \setminus I} x_i \leq B$$

$$b_i = 1, \forall i \in I$$

$$b_i \geq b_j - x_i, \forall (j, i) \in E_\tau$$

$$b_i \leq 1 - x_i, \forall i \in C$$

$$x_i = 0, \forall i \in I \cup T$$

Plain Text

```
# MILP for Model A in AMPL

# Sets
set CELLS;
set IGNITION within CELLS;
set ACTIVE_EDGES within {CELLS, CELLS};
set TARGETS within CELLS;

# Parameters
param weight {TARGETS};
param budget;

# Variables
var clear {i in CELLS} binary;
var burn {i in CELLS} binary;

# Objective - minimise total weighted damage
minimize TotalDamage:
    sum {i in TARGETS} weight[i] * burn[i];

# Budget constraint - limited budget
subject to Budget:
    sum {i in CELLS diff IGNITION} clear[i] <= budget;

# Ignition constraint and don't clear ignition cells
subject to Ignition {i in IGNITION}:
    burn[i] = 1;
subject to NoClearIgnition {i in IGNITION}:
    clear[i] = 0;

# Model A - Binary Propagation
subject to Propagation {(j,i) in ACTIVE_EDGES}:
    burn[i] >= burn[j] - clear[i]; # Burn-clear rule handled below

# Burn-clear rule
subject to DoesNotBurnIfCleared {i in CELLS}:
    burn[i] <= 1 - clear[i];
subject to DoesNotClearTargets {i in TARGETS}:
    clear[i] = 0;
```

R

```
# An edge is "active" if:
```

```r
tau <- 0.1 # Filters low-probability edges below likelihood of meaningful spread
active_edges <- edges[edges$p_burn > tau, c("from_node", "to_node")]

# Weighted target cells
target_cells <- data.frame(
    node = c(settlement_node, wetland_node),
    weight = c(rep(10, nrow(settlement)), rep(5, nrow(wetland)))
)

damage_A <- c()
firebreak_A <- list()
for (a in c(0, 1, 5, 10, 15, 20)) {
     ampl$reset()
    ampl$read("milpmodelA.mod")
    ampl$setOption("solver", "gurobi")
    ampl$getSet("CELLS")$setValues(total_nodes)
    ampl$getSet("IGNITION")$setValues(source_nodes)
    ampl$getSet("ACTIVE_EDGES")$setValues(active_edges)
    ampl$getSet("TARGETS")$setValues(target_cells$node)
    ampl$getParameter("weight")$setValues(target_cells)
    ampl$getParameter("budget")$set(a)
    ampl$solve() # Solve

     # Store damage
    damage <- ampl$getValue("TotalDamage")
     damage_A <- c(damage_A, damage)

     # Store firebreak solution
     clear_values_A <- ampl$getVariable("clear")$getValues()
    cleared_cells_A <- clear_values_A[clear_values_A$clear.val > 0.5, ]
     if (nrow(cleared_cells_A) > 0) {
     cleared_cells_A$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1", cleared_cells_A$index0))
     cleared_cells_A$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1", cleared_cells_A$index0))
     }

     firebreak_A[[as.character(a)]] <- cleared_cells_A
    cat("Total weighted damage with B =", a, "is", damage, "\n")
}

# Calculate shadow price on budget
budget_shadow_price <- ampl$getConstraint("Budget")$dual()
cat("Budget constraint shadow price is", budget_shadow_price, "\n")
```

Add reachability $r_i \in [0,1], p(u,v) = p_{burn}(u \to v)^{\alpha}, \alpha < 1$.
For Model B:

$$\min_{x,b,r} \Sigma_{i \in T} w_i b_i$$
$$\Sigma_{i \in C \setminus I} x_i \leq B$$
$$r_i = 1, b_i = 1, \forall i \in I$$
$$r_i \geq p(j,i) \cdot r_j - M \cdot x_i, \forall (j,i) \in E$$
$$b_i \leq 1 - x_i, \forall i \in C$$
$$b_i \geq r_i, \forall i \in C \setminus I$$
$$x_i = 0, \forall i \in I$$

Plain Text

```
# MILP for Model B in AMPL, milpmodelB.mod

# Sets
set CELLS;
set IGNITION within CELLS;
set EDGES within {CELLS, CELLS};
set TARGETS within CELLS;

# Parameters
param weight {TARGETS};
param budget;
param p {EDGES};
param M := 1;

# Variables
var clear {i in CELLS} binary;
var burn {i in CELLS} binary;
var reach {i in CELLS} >= 0, <= 1;

# Objective - minimise total weighted damage
minimize TotalDamage:
    sum {i in TARGETS} weight[i] * burn[i];

# Budget constraint - limited budget
subject to Budget:
    sum {i in CELLS diff IGNITION} clear[i] <= budget;

# Ignition constraint
subject to Ignition {i in IGNITION}:
    reach[i] = 1;
subject to IgnitionBurn {i in IGNITION}:
    burn[i] = 1;
```

```
subject to NoClearIgnition {i in IGNITION}:
    clear[i] = 0;

# Model B - Continuous Reachability
subject to Propagation {(j,i) in EDGES}:
    reach[i] >= p[j,i] * reach[j] - M * clear[i];

# Burn-clear-reach rule
subject to DoesNotBurnIfCleared {i in CELLS}:
    burn[i] <= 1 - clear[i];
subject to BurnIfReached {i in CELLS diff IGNITION}:
    burn[i] >= reach[i];
```

R

```r
# Compressed probabilities
alpha <- 0.2 # Choose 0.2 as a reasonable definitive value for alpha < 1
edges$p_compressed <- edges$p_burn^alpha

# All edges for Model B
all_edges <- edges[, c("from_node", "to_node", "p_compressed")]

damage_B <- c()
firebreak_B <- list()
for (b in c(0, 1, 5, 10, 15, 20)) {
    ampl$reset()
    ampl$read("milpmodelB.mod")
    ampl$setOption("solver", "gurobi")
    ampl$getSet("CELLS")$setValues(total_nodes)
    ampl$getSet("IGNITION")$setValues(source_nodes)
    ampl$getSet("EDGES")$setValues(all_edges[, c("from_node", "to_node")])
    ampl$getSet("TARGETS")$setValues(target_cells$node)
    ampl$getParameter("weight")$setValues(target_cells)
    ampl$getParameter("budget")$set(b)
    ampl$getParameter("p")$setValues(all_edges)
    ampl$solve()

    damage <- ampl$getValue("TotalDamage")
     damage_B <- c(damage_B, damage)

     clear_values_B <- ampl$getVariable("clear")$getValues()
    cleared_cells_B <- clear_values_B[clear_values_B$clear.val > 0.5, ]

     cleared_cells_B$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1",
cleared_cells_B$index0))
```

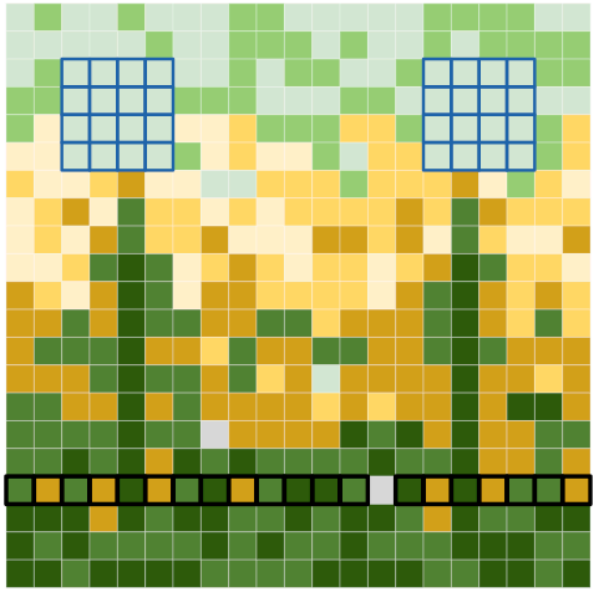
```
    cleared_cells_B$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1",
cleared_cells_B$index0))

    firebreak_B[[as.character(b)]] <- cleared_cells_B
   cat("Total weighted damage with B =", b, "is", damage, "\n")
}

# Calculate shadow price on budget
budget_shadow_price <- ampl$getConstraint("Budget")$dual()
cat("Budget constraint shadow price is", budget_shadow_price, "\n")
```

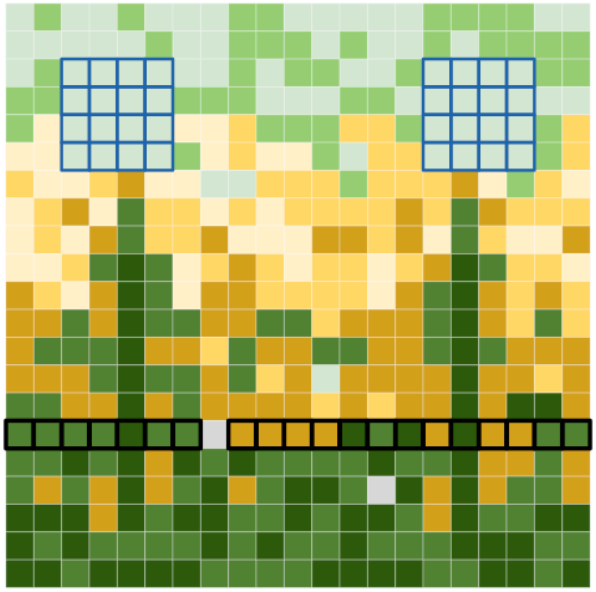
4.2.3
2)

Budget (B)	Model A	Model B
0	240	240
1	240	230
5	240	190
10	240	120
15	80	60
20	0	0

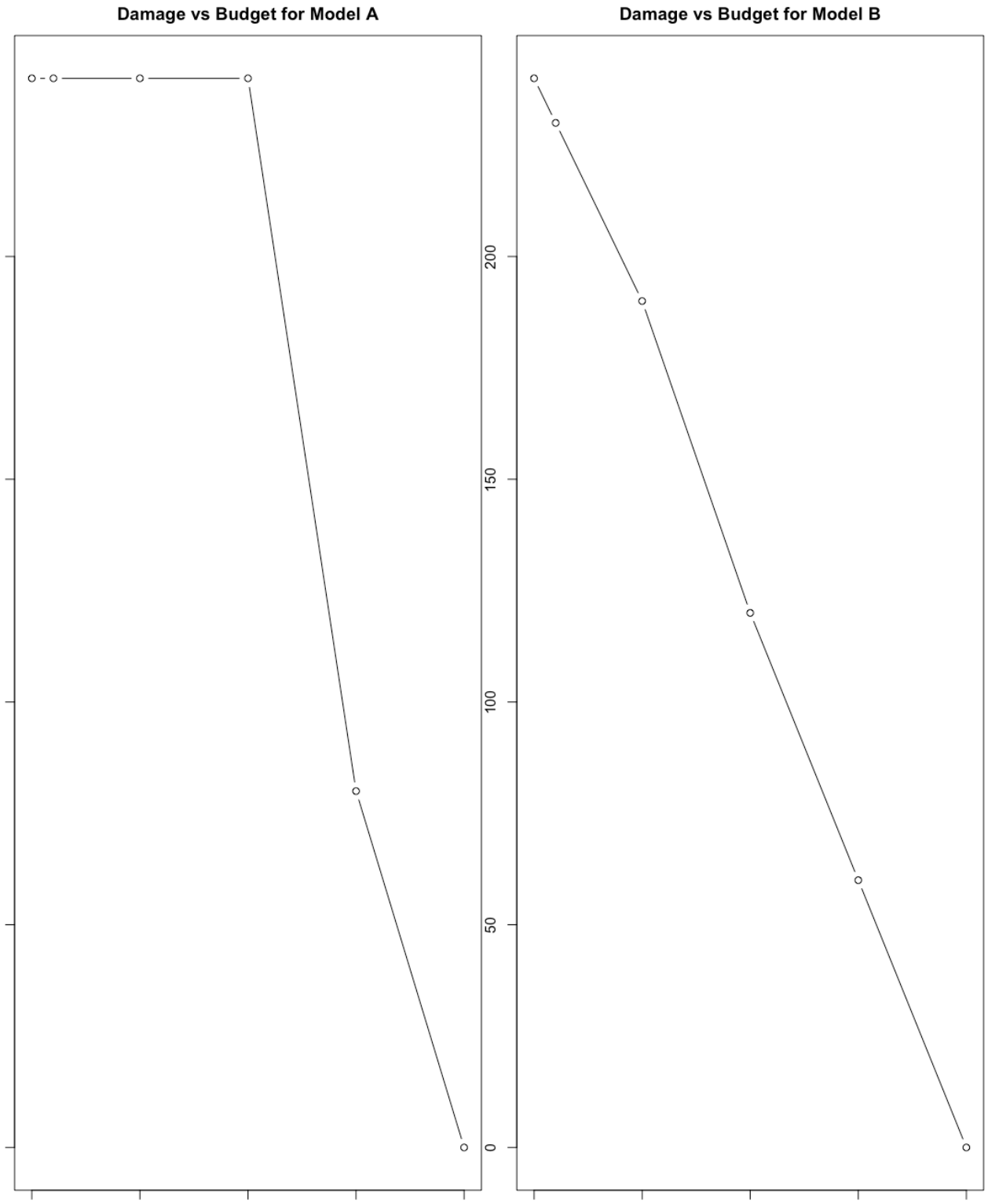
Model A's optimal firebreak locations at B=20

Model B's optimal firebreak locations at B=20





Plotting both as curves for comparison gives:



3) Model A thinks that the firebreak should be across the entire row 18, whilst Model B thinks that the firebreak should be across the entire row 16 (for both, excluding the cell with no vegetation, since this will not burn). With Model A and budget B = 20, we get 0 damage. Hence, adding an extra cell and additional spending beyond this has no effect. With lower values of B, weighted damage is maximised to (16 * 10) + (16 * 5) = 160 + 80 = 240, when B = 0. With binary propagation, until a critical cut is hit, damage is high, but when the cut occurs, damage drops fast towards 0. Shadow prices should not be interpreted as marginal values in A. Continuous LPs are required and the integer constraints mean that the dual values returned are not meaningful economically, so the MILP does not have LP duality applied. The same applies to Model B, where shadow prices are not meaningful since integer constraints prevent LP duality. A meaningful marginal value can be recovered by solving LP relaxation and reading the dual on the budget constraint there.

4) The models disagree on placement, agreeing only that B = 20 achieves zero damage. Model A places its barrier across row 18. This is due to binary propagation and usingτ = 0.1. The model builds the barrier at the closest possible place before the fire reaches the northern half of the grid - the minimum cut. It wants to block off paths completely and as quickly as possible, so places the barrier more south than Model B.
Model B places its barrier across row 16 due to compressed probabilities withα= 0.2, chosen to distinguish reachability across paths whilst preserving ignition ordering. Smallerα flattens differences between cells, whilst largerαapproaches Model A's raw probabilities. Reachability decays along paths due top^{α}, and the model seeks to exploit natural decay. By row 16, the product of compressed probabilities along any path is small, and the model finds it more efficient to cut at row 16, where reachability is impactful enough to be worth blocking.

5) In comparison to the current corridor, both place a horizontal barrier, at rows 18 and 16 for models A and B respectively. Step 2 uses 20 cells vs current corridor's 17, which has 2 cells open on each side. Step 2 achieves 0 damage when using a budget of 20 under both models A and B, yet the corridor may not. Step 1 identifies bottlenecks near targets, but Step 2 places barriers near the ignition front, since early blocking is cheaper under fixed wind and ignition.

6) Running the fire simulator with the optimal firebreaks:

```R
ignition_row <- cbind(rep(21,21), 1:21) # Row 21; all cells ignited
corridor <- cbind(rep(10, 17), 3:19) # Current corridor

landscape_model_A <- set_firebreaks(landscape,
                                    as.matrix(firebreak_A[["20"]][, c("row", "col")]))
landscape_model_B <- set_firebreaks(landscape,
                                    as.matrix(firebreak_B[["20"]][, c("row", "col")]))
landscape_corridor <- set_firebreaks(landscape, corridor)

set.seed(1) # Makes results reproducible
# Gets average damage for 50 simulations, using helper functions in fire-simulator.r
damage_simulator_A <- replicate(50, {
    simulator <- simulate_fire(landscape_model_A, ignition_row, wind_speed=0, wind_dir=0)
    compute_damage(simulator$burned, targets)
})

set.seed(1)
damage_simulator_B <- replicate(50, {
    simulator <- simulate_fire(landscape_model_B, ignition_row, wind_speed=0, wind_dir=0)
```

```
    compute_damage(simulator$burned, targets)
})

set.seed(1)
damage_simulator_corridor <- replicate(50, {
    simulator <- simulate_fire(landscape_corridor, ignition_row,
wind_speed=0, wind_dir=0)
    compute_damage(simulator$burned, targets)
})

set.seed(1)
damage_simulator_base <- replicate(50, {
    simulator <- simulate_fire(landscape, ignition_row, wind_speed=0,
wind_dir=0)
    compute_damage(simulator$burned, targets)
})

cat("Model A with B = 20 mean simulated damage:", mean(damage_simulator_A),
"\n")
cat("Model B with B = 20 mean simulated damage:", mean(damage_simulator_B),
"\n")
cat("Corridor mean simulated damage:", mean(damage_simulator_corridor), "\n")
cat("Baseline mean simulated damage:", mean(damage_simulator_base), "\n")
```

Overall results:

Method	Model prediction	Simulator
Model A, B=20	0	0
Model B, B=20	0	0
Corridor	N/A	0.6
Baseline	N/A	16

The simulated damage matches the prediction of both models. A model can predict poorly but produce a good plan. Here, both predict the same since the simulator matches the scenario. With wind conditions, plan quality matters more than prediction accuracy

4.2.4

1) Both models predict well and produce good plans since the simulator matches the exact optimal scenario. These can diverge however; a model predicting 0 damage by clearing target cells would score well on prediction but produces an unusable plan, which is a limitation of Model B. Adding

explicit target-clearing constraints recovers correctness, but removes the smooth reachability progression that distinguishes Model B from Model A.

2) Both models agree that B=20 is where spending stops helping prevent damage to the weighted cells, since damage reaches 0. The authority needs to budget 20 cells minimum to fully protect the settlement and the wetland under the fixed no wind scenario.

3) This plan performs poorly under south west wind (confirmed in Step 3). The worst 10% of scenarios occur at wind direction -0.86 radians with ignition around rows 10-11, columns 9-10. These are conditions where fire bypasses the horizontal barrier entirely. Under uniform random ignition, any fire starting north of row 16 is also unaffected by the barrier.

4) The plan would change since settlement is weighted 2x more than the wetland, so that the firebreak is placed closer to the targets. If there were equal damage weights, the model would treat the settlement and wetland cells identically, shifting firebreak cells towards the ignition row and reducing the probability of fire reaching them. The models implicitly prioritise the settlement; clearing cells that block fire to the settlement is worth more than clearing the cells blocking fire to the wetland.

Step 3: Evaluation under uncertain wind
Choose Model B: more realistic since it accounts for multiplicative reachability probabilities and fire pathway strength.

```R
# Model B, B=13
damage_B_B13 <- c()
ampl$reset()
ampl$read("milpmodelB.mod")
ampl$setOption("solver", "gurobi")
ampl$getSet("CELLS")$setValues(total_nodes)
ampl$getSet("IGNITION")$setValues(source_nodes)
ampl$getSet("EDGES")$setValues(all_edges[, c("from_node", "to_node")])
ampl$getSet("TARGETS")$setValues(target_cells$node)
ampl$getParameter("weight")$setValues(target_cells)
ampl$getParameter("budget")$set(13)
ampl$getParameter("p")$setValues(all_edges)
ampl$solve() # Solve

# Store damage
damage_B13 <- ampl$getValue("TotalDamage")
damage_B_B13 <- c(damage_B_B13, damage_B13)

# Store firebreak solution
clear_values_B_B13 <- ampl$getVariable("clear")$getValues()
cleared_cells_B_B13 <- clear_values_B_B13[clear_values_B_B13$clear.val > 0.5,
]

```

```
cleared_cells_B_B13$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1",
cleared_cells_B_B13$index0))
cleared_cells_B_B13$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1",
cleared_cells_B_B13$index0))

firebreak_B[["13"]] <- cleared_cells_B_B13
cat("Total weighted damage with B = 13 is", damage_B13, "\n")
landscape_B_B13 <- set_firebreaks(landscape, as.matrix(cleared_cells_B_B13[,
c("row", "col")]))
# We get 70 damage for Model B
```

Wind speed follows$V$ ~ Weibull$(k = 2, A = 7)$.
We want to sample and invert from its cumulative distribution function:

$$F_V(v) = 1 - e^{(-\frac{v}{A})^k}$$

which gives$v = A(-\log(1-U))^{\frac{1}{k}}$, U ~ Uniform(0, 1) .

R

```
#' Sample wind speed from Weibull
#'
#' Uses inverse transform to sample from Weibull distribution
#'
#' @param n No. of samples
#' @param k Shape parameter (2 for wind speed)
#' @param A Scale parameter in km/h (7 default)
#' @return Vector of wind speed in km/h
wind_speed_sampler <- function(n, k=2, A=7) {
    U <- runif(n)
    A * (-log(1-U))^(1/k)
}

set.seed(1)
speeds <- wind_speed_sampler(1000)
cat("Mean wind speed is", mean(speeds), "km/h\n") # 6.23km/h
# E[V] = A * gamma(1 + 1/k)
cat("Expected mean is", 7 * gamma(1 + 1/2), "\n" )
```

Wind direction follows a mixed von Mises distribution, with wind probability 0.7,$\mu_1 = 1.5\pi, \kappa_1 = 3$ from the west, and with wind probability 0.3,$\mu_2 = 0.75\pi, \kappa_2 = 2$from the south-east:

$$f_\Theta(\theta) = 0.7 \cdot \frac{e^{3\cos(\theta - 1.5\pi)}}{2\pi I_0(3)} + 0.3 \cdot \frac{e^{2\cos(\theta - 0.75\pi)}}{2\pi I_0(2)}$$

Use acceptance-rejection as the von Mises distribution CDF has no closed form and is semi continuous on$[-\pi, \pi]$,$g_\Theta(\theta) = \frac{1}{2\pi}$,$M = \max_\kappa\left\{\frac{f_\Theta(\theta)}{g_\Theta(\theta)}\right\} = \max_\kappa\{f_\Theta(\theta) \cdot 2\pi\}$.

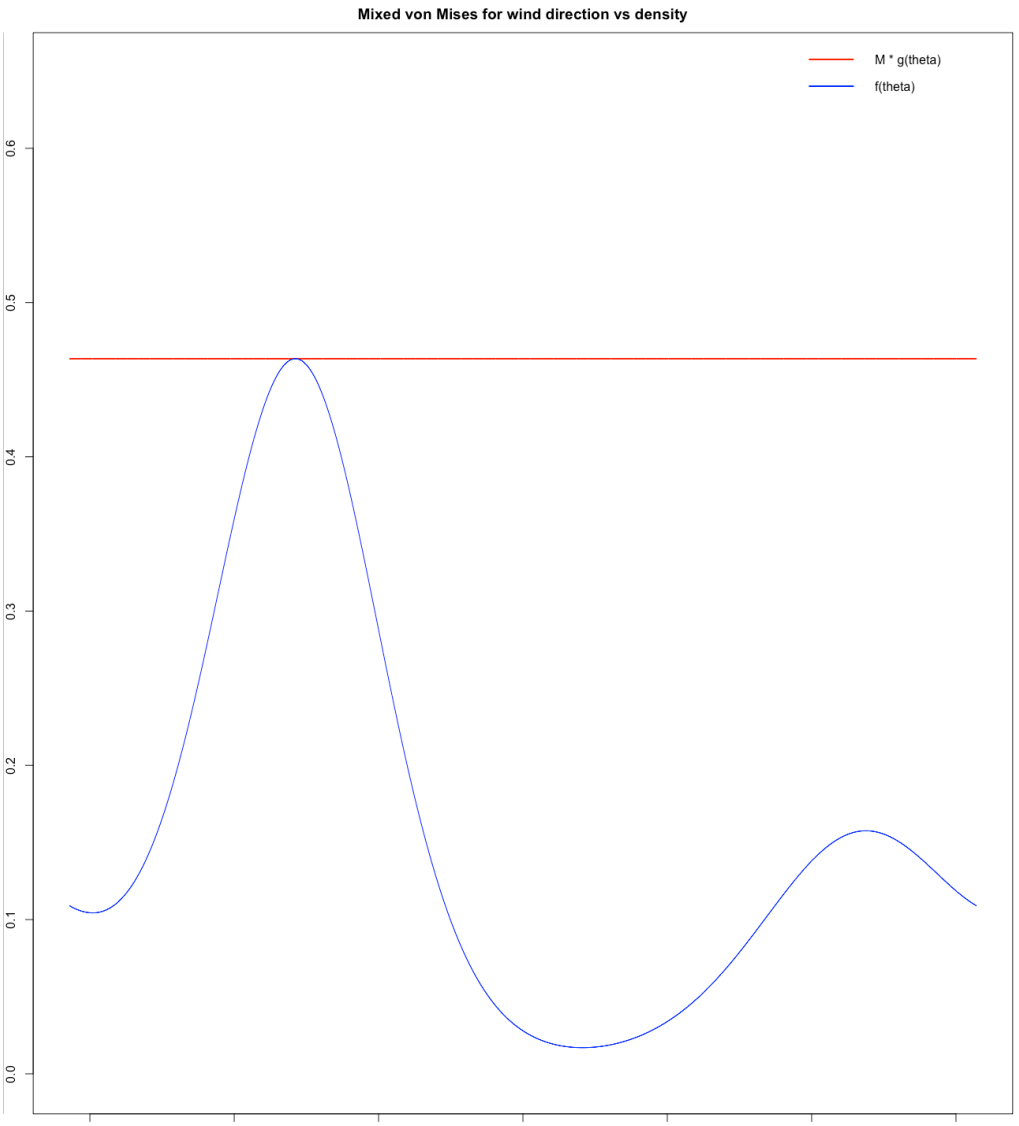
1. Sample$Y$ from Uniform$(-\pi, \pi)$
2. Sample$U$ ~ Uniform(0, 1)
3. If$U \leq \frac{f_\Theta(\theta)}{M \cdot g_\Theta(\theta)}$, accept$Y$. Otherwise, go to step 1.

Lignition is sample uniformly over all vegetated cells.

```R
#' Find M for mixed von Mises density
#'
#' @param theta Vector of angle in radians
#' @param mu_1 Mean of first component of mixed von Mises
#' @param mu_2 Mean of second component of mixed von Mises
#' @param kappa_1 Concentration of first component
#' @param kappa_2 Concentration of second component
#' @return Vector of density values
#'
#' @examples
#' mixed_vonmises(seq(-pi, pi, length.out=100))
mixed_vonmises <- function(theta, mu_1 = 1.5*pi, mu_2 = 0.75*pi, kappa_1 = 3, kappa_2 = 2) {
    f_1 <- exp(kappa_1 * cos(theta - mu_1)) / (2 * pi * besselI(kappa_1, nu=0))
    f_2 <- exp(kappa_2 * cos(theta - mu_2)) / (2*pi* besselI(kappa_2, nu=0))
    0.7 * f_1 + 0.3 * f_2
}

M <- max(mixed_vonmises(seq(-pi, pi, length.out = 10000))) * 2 * pi
cat("M =", M, "\n")

#' Sample wind direction from mixed von Mises density
#'
#' Use acceptance-rejection, proposing Uniform(-pi, pi) as the distribution
#'
#' @param n No. of samples
#' @param mu_1 Mean of first component of mixed von Mises
#' @param mu_2 Mean of second component of mixed von Mises
#' @param kappa_1 Concentration of first component
#' @param kappa_2 Concentration of second component
#' @return Vector of wind directions in radians
#'
#' @examples
```

```r
#' set.seed(1)
#' wind_direction_sampler(100)$acceptance_rate
wind_direction_sampler <- function(n, mu_1 = 1.5*pi, mu_2 = 0.75*pi, kappa_1 = 3, kappa_2 = 2) {
    M <- max(mixed_vonmises(seq(-pi, pi, length.out = 10000), mu_1, mu_2, kappa_1, kappa_2)) * 2 * pi
    accepted <- numeric(n)
    num_proposals <- 0
    num_accepted <- 0

    # Method 2: Sampling ahead of time
    while (num_accepted < n) {
        batch_size <- ceiling((n - num_accepted) * M * 1.1)
        y <- runif(batch_size, -pi, pi) # Sample from Y = g_Theta (theta)
        u <- runif(batch_size) # Sample from U ~ Uniform(0, 1)
        accept_prob <- mixed_vonmises(y, mu_1, mu_2, kappa_1, kappa_2) * 2 * pi / M
        keep <- u <= accept_prob
        new_accepted <- sum(keep)
        num_proposals <- num_proposals + batch_size
        # Step 3: Accept Y
        if (new_accepted > 0) {
            take <- min(new_accepted, n - num_accepted)
            accepted[(num_accepted + 1):(num_accepted + take)] <- y[keep][1:take]
            num_accepted <- num_accepted + take
        }
    }
    # Acceptance rate is the number of samples / number of proposals
    list(samples = accepted, acceptance_rate = n / num_proposals)
}

# Test acceptance rate
set.seed(1)
directions <- wind_direction_sampler(1000)
cat("Acceptance rate is", directions$acceptance_rate, "\n" )
```

Acceptance rate is 0.3120125 (≈ 31%), close to the theoretical value of accepting 1/M = 0.3433159 (≈34%).



The envelope condition is satisfied; the red line ($M \cdot g_\Theta(\theta)$) is always above the blue line ($f_\Theta(\theta)$), $\forall\theta . M$ will be the tightest bound for $\frac{f_\Theta(\theta)}{g_\Theta(\theta)}$, which is a requirement for a valid acceptance-rejection method, as the red line has a very small gap to the target blue line. This means that the distribution we proposed is efficient for the mixed distribution, and choosing a different M would give us a lower acceptance rate.

```R
#' Sample location of ignition uniformly over vegetated cells
#'
#' Finds vegetated cells from all cells and picks a random cell uniformly
#'
#' @param landscape Landscape matrix
#' @return Vector of length 2, c(row, col), which is the location of ignition
#'
#' @examples
#' set.seed(1)
#' ignition_sampler(landscape)
ignition_sampler <- function(landscape) {
    vegetated <- which(landscape > 0, arr.ind = TRUE)
```

```
    idx <- sample(nrow(vegetated), 1)
    vegetated[idx, ]
}

set.seed(1)
ignition <- ignition_sampler(landscape)
cat("Ignited on row", ignition[1], ", col", ignition[2], "\n")
```

R

```
#' Run Monte Carlo to evaluate firebreak plan
#'
#' Use the three samplers for each scenario, recording weighted damage and #' scenario metadata
#'
#' @param landscape_firebreaks Landscape matrix with firebreaks
#' @param targets Data frame with target cells and their weights
#' @param N no. of simulated scenarios (1000 for long term behaviour)
#' @return Data frame of damages, wind_speed, wind_direction, ignition_row, #'    ignition_col in each scenario
#' @examples
#' set.seed(1)
#' monte_carlo(landscape_B_B13, targets, N=100)
monte_carlo <- function(landscape_firebreaks, targets, N = 1000) {
    scenario_df <- data.frame(
      damage = numeric(N),
       wind_speed = numeric(N),
      wind_direction = numeric(N),
      ignition_row = numeric(N),
      ignition_col = numeric(N)
   )
    for (i in 1:N) {
        wind_speed <- wind_speed_sampler(1)
        wind_direction <- wind_direction_sampler(1)$samples
        ignition <- ignition_sampler(landscape_firebreaks)
        simulation <- simulate_fire(landscape_firebreaks, ignition, wind_speed, wind_direction)
        scenario_df$damage[i] <- compute_damage(simulation$burned, targets)
        scenario_df$wind_speed[i] <- wind_speed
        scenario_df$wind_direction[i] <- wind_direction
        scenario_df$ignition_row[i] <- ignition[1]
        scenario_df$ignition_col[i] <- ignition[2]
    }
    scenario_df
}
```

```r

set.seed(1)
N <- 1000
result_MC_B13 <- monte_carlo(landscape_B_B13, targets, N)
damage_MC_B13 <- result_MC_B13$damage

set.seed(1)
result_MC_corridor <- monte_carlo(landscape_corridor, targets, N)
damage_MC_corridor <- result_MC_corridor$damage

set.seed(1)
result_MC_baseline <- monte_carlo(landscape, targets, N)
damage_MC_baseline <- result_MC_baseline$damage

# Report results
confidence <- 0.95
estimate <- mean(damage_MC_B13)
empirical_std_error <- sd(damage_MC_B13) / sqrt(N)
t_critical <- qt((1+confidence)/2, df = N-1)
ci_lower <- estimate - t_critical * empirical_std_error
ci_upper <- estimate + t_critical * empirical_std_error

cat(paste0("Monte Carlo estimate for mean damage is ", estimate, "\n",
           "The 95% confidence interval is [", ci_lower, ", ", ci_upper,
"]\n"))

# Report results
corridor_estimate <- mean(damage_MC_corridor)
corridor_ese <- sd(damage_MC_corridor) / sqrt(N)
corridor_ci_lower <- corridor_estimate - t_critical * corridor_ese
corridor_ci_upper <- corridor_estimate + t_critical * corridor_ese

cat(paste0("Monte Carlo estimate for mean damage is ", corridor_estimate,
"\n",
           "The 95% confidence interval is [", corridor_ci_lower, ", ",
corridor_ci_upper, "]\n"))

# Report results with 95% confidence intervals
baseline_estimate <- mean(damage_MC_baseline)
baseline_ese <- sd(damage_MC_baseline) / sqrt(N)
baseline_ci_lower <- baseline_estimate - t_critical * baseline_ese
baseline_ci_upper <- baseline_estimate + t_critical * baseline_ese

cat(paste0("Monte Carlo estimate for mean damage is ", baseline_estimate,
"\n",
           "The 95% confidence interval is [", baseline_ci_lower, ", ",
baseline_ci_upper, "]\n"))
```

```r

#' Estimate CVaR at level alpha
#'
#' Computes the Conditional Value at Risk as the mean of samples
#' at or above the alpha-quantile, with standard error
#'
#' @param x Numeric vector of loss samples
#' @param alpha Quantile level in (0, 1)
#' @return List with `estimate` (scalar), `se` (scalar), and `n` (tail size)
#' @examples
#' estimate_cvar(rexp(1000, 1/200), 0.95)
estimate_cvar <- function(x, alpha) {
    threshold <- quantile(x, alpha)
    tail <- x[x >= threshold]
    list(
      estimate = mean(tail),
      se = sd(tail) / sqrt(length(tail)),
      n = length(tail)
    )
}

# Calculate CVaRs of methods, alpha = 0.9
cvar_B_B13 <- estimate_cvar(damage_MC_B13, 0.9)
cvar_corridor <- estimate_cvar(damage_MC_corridor, 0.9)
cvar_baseline <- estimate_cvar(damage_MC_baseline, 0.9)

cat("CVaR 0.9 of Model B, B=13 is", cvar_B_B13$estimate, "\n")
cat("CVaR 0.9 of corridor is", cvar_corridor$estimate, "\n")
cat("CVaR 0.9 of baseline is", cvar_baseline$estimate, "\n")

# CVaR estimates with 95% confidence intervals
t_critical_B_B13 <- qt((1+confidence)/2, df = cvar_B_B13$n - 1)
t_critical_corridor <- qt((1+confidence)/2, df = cvar_corridor$n - 1)
t_critical_baseline <- qt((1+confidence)/2, df = cvar_baseline$n - 1)
cat("95% confidence interval for CVaR 0.9 Model B, B=13 is [",
cvar_B_B13$estimate - t_critical_B_B13 * cvar_B_B13$se, ", ",
cvar_B_B13$estimate + t_critical_B_B13 * cvar_B_B13$se, "]\n")
cat("95% confidence interval for CVaR 0.9 corridor is [",
cvar_corridor$estimate - t_critical_corridor * cvar_corridor$se, ", ",
cvar_corridor$estimate + t_critical_corridor * cvar_corridor$se, "]\n")
cat("95% confidence interval for CVaR 0.9 baseline is [",
cvar_baseline$estimate - t_critical_baseline * cvar_baseline$se, ", ",
cvar_baseline$estimate + t_critical_baseline * cvar_baseline$se, "]\n")
```

Overall results:

Method	Expected mean damage (EMD)	95% Confidence Interval of EMD	$CVaR_{0.9}$	95% Confidence Interval of $CVaR_{0.9}$
Model B, B=13	3.165	[2.73, 3.60]	17.06	[15.61, 18.51]
Current corridor	3.76	[3.05, 4.47]	24.32	[21.05, 27.58]
Baseline	13.84	[12.56, 15.12]	57.92	[54.59, 61.24]

4.3.4

1)

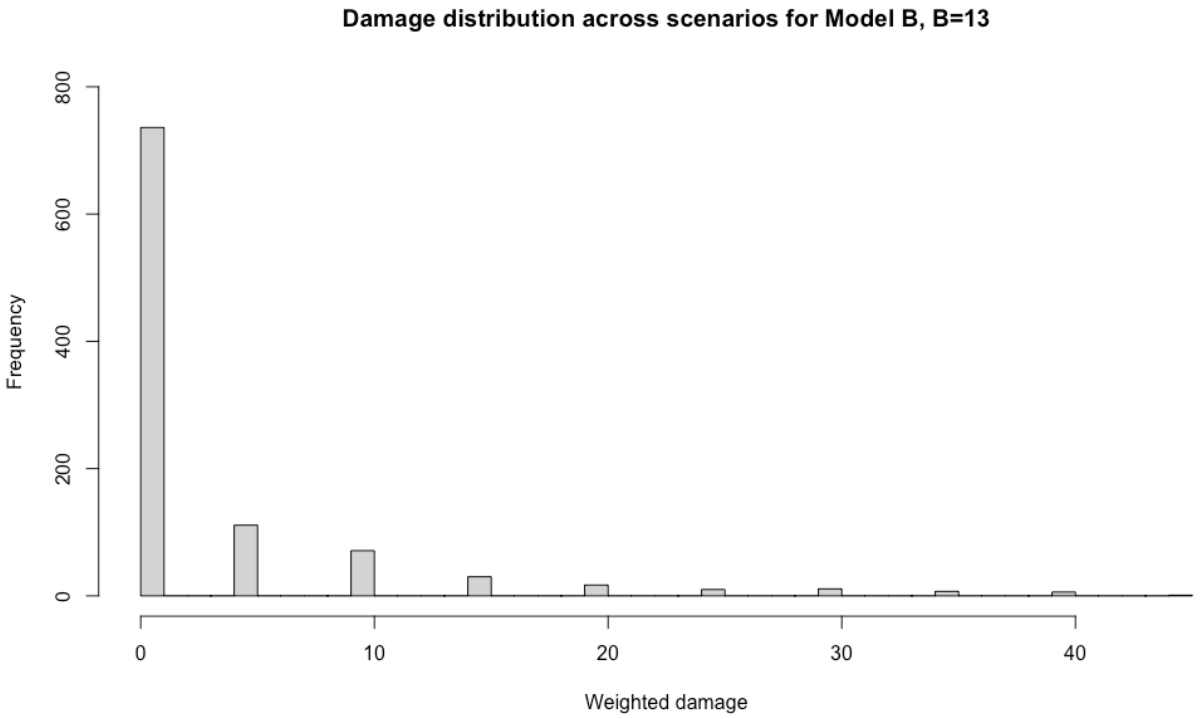
```R
# Find worst 10% of scenarios in Model B, B=13
limit <- quantile(damage_MC_B13, 0.9)
worst <- result_MC_B13[result_MC_B13$damage >= limit, ]

cat("Mean wind speed in the worst 10% is", mean(worst$wind_speed), "\n")
cat("Mean wind direction in the worst 10% is", mean(worst$wind_direction), "\n")
cat("Mean ignition row and col in the worst 10% is", mean(worst$ignition_row), ",", mean(worst$ignition_col), "\n")
```

The worst 10% of scenarios occur when wind direction is -0.86 radians (south-west), pushing fire north-east, towards the settlement cells, and ignition at rows 10-11, columns 9-10. Wind speed is 6.17km/h, close to the Weibull sampler mean (6.23 km/h), confirming wind direction causes worst scenarios more than speed. The deterministic plan fails when wind pushes fire around the firebreak to reach targets from the side.

2) Model B, B=13 achieves lower expected mean damage (3.165 vs 3.76) and lower $CVaR_{0.9}$ (17.06 vs 24.32) than the current corridor. Target placement beats broad coverage - Model B concentrates firebreak cells where fire pressure is high, whilst the corridor spreads the budget across the landscape for broad coverage.

3) The plan would likely fail if we knew wind conditions in advance. The barrier across the landscape is only effective with southern ignition and fire travelling north. Under conditions where wind comes at an angle (like south-west or south-east), fire will spread diagonally north-east or north-west around the barrier, and reach the settlement and wetland cells. If we had wind knowledge, the barrier would most likely shift diagonally north-east towards columns 14-17. The current plan is not optimal for more complex conditions that create the worst possible scenarios.

4)



Distribution is right skewed, where the majority of scenarios have little to no damage, but a small number of scenarios produce high amounts of damage. Expected damage understates risk, since most scenarios have zero damage, pulling the mean down. $\text{CVaR}_{0.9}$better captures the 10% worst case scenarios that matter most for protecting the targets.

5) We use N=1000, which, for the 95% confidence interval is [2.73, 3.60], 3.60 - 2.73 = 0.87 width. The right skewed distribution increases variance, so the choice of N depends on required precision and cost.

Step 4: Reactive Response

Same as Model B with additional set$P$permanent cells.

$x_i \in \{0,1\}$for only$C \setminus P \setminus I$

$$\Sigma_{i \in C \setminus P \setminus I} x_i \leq R$$

$$r_i \geq p\,(j,i) \cdot r_j - M \cdot x_i, \forall\,(j,i) \in E, i \notin P, i \notin I$$
$$b_i = 0, \forall i \in P$$

```R
# Solve MILP for B = 17
damage_B_B17 <- c()
ampl$reset()
ampl$read("milpmodelB.mod")
ampl$setOption("solver", "gurobi")
ampl$getSet("CELLS")$setValues(total_nodes)
ampl$getSet("IGNITION")$setValues(source_nodes)
ampl$getSet("EDGES")$setValues(all_edges[, c("from_node", "to_node")])
ampl$getSet("TARGETS")$setValues(target_cells$node)
ampl$getParameter("weight")$setValues(target_cells)
ampl$getParameter("budget")$set(17)
```

```r
ampl$getParameter("p")$setValues(all_edges)
ampl$solve()

# Store damage
damage_B17 <- ampl$getValue("TotalDamage")
damage_B_B17 <- c(damage_B_B17, damage_B17)

# Store firebreak solution
clear_values_B_B17 <- ampl$getVariable("clear")$getValues()
cleared_cells_B_B17 <- clear_values_B_B17[clear_values_B_B17$clear.val > 0.5,
]

cleared_cells_B_B17$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1",
cleared_cells_B_B17$index0))
cleared_cells_B_B17$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1",
cleared_cells_B_B17$index0))

firebreak_B[["17"]] <- cleared_cells_B_B17
cat("Total weighted damage with B = 17 is", damage_B17, "\n")
landscape_B_B17 <- set_firebreaks(landscape, as.matrix(cleared_cells_B_B17[,
c("row", "col")]))

# Solve MILP for B = 12
damage_B_B12 <- c()
ampl$reset()
ampl$read("milpmodelB.mod")
ampl$setOption("solver", "gurobi")
ampl$getSet("CELLS")$setValues(total_nodes)
ampl$getSet("IGNITION")$setValues(source_nodes)
ampl$getSet("EDGES")$setValues(all_edges[, c("from_node", "to_node")])
ampl$getSet("TARGETS")$setValues(target_cells$node)
ampl$getParameter("weight")$setValues(target_cells)
ampl$getParameter("budget")$set(12)
ampl$getParameter("p")$setValues(all_edges)
ampl$solve()

# Store damage
damage_B12 <- ampl$getValue("TotalDamage")
damage_B_B12 <- c(damage_B_B12, damage_B12)

# Store firebreak solution
clear_values_B_B12 <- ampl$getVariable("clear")$getValues()
cleared_cells_B_B12 <- clear_values_B_B12[clear_values_B_B12$clear.val > 0.5,
]

cleared_cells_B_B12$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1",
cleared_cells_B_B12$index0))
```

```
cleared_cells_B_B12$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1",
cleared_cells_B_B12$index0))

firebreak_B[["12"]] <- cleared_cells_B_B12
cat("Total weighted damage with B = 12 is", damage_B12, "\n")
landscape_B_B12 <- set_firebreaks(landscape, as.matrix(cleared_cells_B_B12[,
c("row", "col")]))
```

Plain Text

```
# MILP for Model B in AMPL, reactivemilpmodelB.mod

# Sets
set CELLS;
set IGNITION within CELLS;
set EDGES within {CELLS, CELLS};
set TARGETS within CELLS;
set PERMANENT within CELLS;

# Parameters
param weight {TARGETS};
param reactive_budget;
param p {EDGES};
param M := 1;

# Variables
var clear {i in CELLS diff PERMANENT diff IGNITION} binary;
var burn {i in CELLS} binary;
var reach {i in CELLS} >= 0, <= 1;

# Objective - minimise total weighted damage
minimize TotalDamage:
    sum {i in TARGETS} weight[i] * burn[i];

# Budget constraint - limited budget
subject to Budget:
    sum {i in CELLS diff PERMANENT diff IGNITION} clear[i] <=
reactive_budget;

# Ignition constraint
subject to Ignition {i in IGNITION}:
    reach[i] = 1;
subject to IgnitionBurn {i in IGNITION}:
    burn[i] = 1;

```

```
# Model B - Continuous reachability
subject to Propagation {(j,i) in EDGES : i not in PERMANENT and i not in IGNITION}:
    reach[i] >= p[j,i] * reach[j] - M * clear[i];

# Burn-clear-reach
subject to DoesNotBurnCleared {i in CELLS diff PERMANENT diff IGNITION}:
    burn[i] <= 1 - clear[i];
subject to BurnIfReached {i in CELLS diff IGNITION diff PERMANENT}:
    burn[i] >= reach[i];
subject to DoesNotBurnPermanent {i in PERMANENT}:
    burn[i] = 0;
subject to DoesNotClearTargets {i in TARGETS diff PERMANENT diff IGNITION}:
    clear[i] = 0;
```

R

```r
#' Solve reactive MILP for each wind scenario
#'
#' Edge weights become wind-dependent and solve MILP to find R reactive
#' firebreak cells
#'
#' @param landscape_permanent Landscape with permanent firebreak cells
#' @param permanent Data frame (row, col) of permanent firebreak cells
#' @param wind_speed Revealed in km/h
#' @param wind_direction Revealed in radians
#' @param R No. of reactive cells added to budget (4 is default)
#' @return Data frame with row and col of reactively chosen firebreak cells
#'
#' @examples
#' set.seed(1)
#' reactive_solver(landscape_B_B13, cleared_cells_B_B13, wind_speed = 5.2,
#' wind_direction = -0.8, R=4)
reactive_solver <- function(landscape_permanent, permanent_cells, wind_speed, wind_direction, R = 4) {
    # Wind-dependent edge calculations
    wind_edges <- spread_probabilities(landscape_permanent, wind_speed, wind_direction)
    wind_edges$from_node <- paste0("r", wind_edges$from_row, "_c", wind_edges$from_col)
    wind_edges$to_node <- paste0("r", wind_edges$to_row, "_c", wind_edges$to_col)

    # Compressed probabilities and permanent nodes
    wind_edges$p_compressed <- wind_edges$p_burn^alpha
```

```
    all_wind_edges <- wind_edges[, c("from_node", "to_node", "p_compressed")]
    permanent_nodes <- paste0("r", permanent_cells$row, "_c",
permanent_cells$col)

    ampl$reset()
    ampl$read("reactivemilpmodelB.mod")
    ampl$setOption("solver", "gurobi")
    ampl$setOption("gurobi_options", "timelimit=120")
    ampl$getSet("CELLS")$setValues(total_nodes)
    ampl$getSet("IGNITION")$setValues(source_nodes)
    ampl$getSet("EDGES")$setValues(all_wind_edges[, c("from_node",
"to_node")])
    ampl$getSet("TARGETS")$setValues(target_cells$node)
    ampl$getSet("PERMANENT")$setValues(permanent_nodes)
    ampl$getParameter("weight")$setValues(target_cells)
    ampl$getParameter("reactive_budget")$set(R)
    ampl$getParameter("p")$setValues(all_wind_edges)
    ampl$solve()
    # Store reactive firebreak solution
    clear_values_reactive <- ampl$getVariable("clear")$getValues()
    reactive_cells <- clear_values_reactive[clear_values_reactive$clear.val >
0.5, ]
    if (nrow(reactive_cells) > 0) {
    reactive_cells$row <- as.numeric(sub("r([0-9]+)_c[0-9]+", "\\1",
reactive_cells$index0))
    reactive_cells$col <- as.numeric(sub("r[0-9]+_c([0-9]+)", "\\1",
reactive_cells$index0))
    }
 reactive_cells
}
```

R

```
# Test on N scenarios
reactive_cells <- matrix(0, nrow=21, ncol=21)
set.seed(1)
N <- 1000
evaluation <- data.frame(
    damage_no_firebreaks = numeric(N),
    damage_permanent_B13 = numeric(N),
    damage_permanent_B17 = numeric(N),
    damage_reactive = numeric(N)
)

for (i in 1:N) {
```

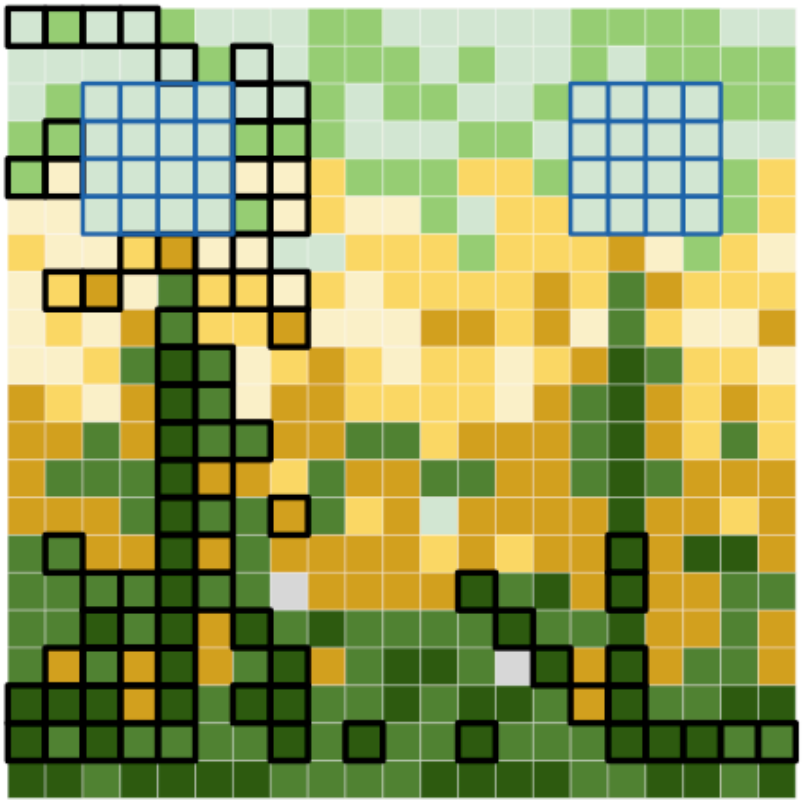
```
    speed <- wind_speed_sampler(1)
    direction <- wind_direction_sampler(1)$samples
    ignition <- ignition_sampler(landscape)

    # Skips firebreak cells
    while (landscape_B_B13[ignition[1], ignition[2]] == 0 | landscape_B_B17[ignition[1], ignition[2]] == 0) {
        ignition <- ignition_sampler(landscape)
    }

    # No firebreaks
    simulation_none <- simulate_fire(landscape, ignition, speed, direction)
    evaluation$damage_no_firebreaks[i] <- compute_damage(simulation_none$burned, targets)
    # B=13
    simulation_B13 <- simulate_fire(landscape_B_B13, ignition, speed, direction)
    evaluation$damage_permanent_B13[i] <- compute_damage(simulation_B13$burned, targets)
    # B+R=17
    simulation_B17 <- simulate_fire(landscape_B_B17, ignition, speed, direction)
    evaluation$damage_permanent_B17[i] <- compute_damage(simulation_B17$burned, targets)
    # Reactive
    reactive <- reactive_solver(landscape_B_B13, cleared_cells_B_B13, speed, direction, R=4)
    if (nrow(reactive) > 0) {
        landscape_reactive <- set_firebreaks(landscape_B_B13, as.matrix(reactive[, c("row", "col")]))
    } else {
        landscape_reactive <- landscape_B_B13
    }

    # Check if ignition cells not cleared by reactive firebreaks
    if (landscape_reactive[ignition[1], ignition[2]] == 0){
        landscape_reactive <- landscape_B_B13
    }

    simulation_reactive <- simulate_fire(landscape_reactive, ignition, speed, direction)
    # Tracks which cells are chosen as reactive cells
    if (nrow(reactive) > 0) {
        for (j in 1:nrow(reactive)) {
            reactive_cells[reactive$row[j], reactive$col[j]] <-
             reactive_cells[reactive$row[j], reactive$col[j]] + 1
        }
```

```
    }
    evaluation$damage_reactive[i] <- compute_damage(simulation_reactive$burned, targets)
}

cat("No firebreaks mean damage", mean(evaluation$damage_no_firebreaks), "\n")
cat("Model B, B=13 mean damage", mean(evaluation$damage_permanent_B13), "\n")
cat("Model B, B=17 mean damage", mean(evaluation$damage_permanent_B17), "\n")
cat("Reactive mean damage", mean(evaluation$damage_reactive), "\n")

# Compute CVaR and confidence intervals
evaluation_plans <- list(
   no_firebreaks = evaluation$damage_no_firebreaks,
   permanent_B13 = evaluation$damage_permanent_B13,
   permanent_B17 = evaluation$damage_permanent_B17,
   reactive = evaluation$damage_reactive
)

for (plan in names(evaluation_plans)) {
    x <- evaluation_plans[[plan]]
    estimate <- mean(x)
    standard_error <- sd(x) / sqrt(N)
    t_c <- qt((1+confidence)/2, df=N-1)
    cvar <- estimate_cvar(x, 0.9)
    t_cvar <- qt((1+confidence)/2, df=cvar$n-1)
    cat(plan, "has mean:", estimate, "with confidence interval: [", estimate-t_c*standard_error, ",", estimate+t_c*standard_error, "]\n")
    cat(" and has CVaR 0.9:", cvar$estimate, "with confidence interval [", cvar$estimate - t_cvar * cvar$se, ",", cvar$estimate + t_cvar * cvar$se, "]\n")
}

reactive_gap <- evaluation$damage_permanent_B13 - evaluation$damage_reactive
cat("Mean reactive effort gap is", mean(reactive_gap), "\n")
cat("Minimum reactive effort gap is", min(reactive_gap), "\n")
cat("Maximum reactive effort gap is", max(reactive_gap), "\n")
cat("Helpful reactive scenarios:", sum(reactive_gap > 0) / N * 100, "%\n")
cat("Hindered reactive scenarios:", sum(reactive_gap < 0) / N * 100, "%\n")
```

Overall results:

Method	Mean damage (N=20, timelimit=60)	Mean damage (N = 1000, timelimit=120)	95% Confidence Interval	$CVaR_{0.9}$	CVaR 95% Confidence Interval
No firebreaks (1)	0.375	12.67	[11.34, 14.00]	61.44	[56.61, 66.27]
Model B, B=13 (2)	0.105	2.6	[2.23, 2.97]	15.66	[14.44, 16.88]
Model B, B+R=17 (3)	0.03	1.73	[1.43, 2.02]	9.76	[8.69, 10.80]
Reactive B=13, R=4 (4)	0.035	2.46	[2.08, 2.84]	16.42	[15.03, 17.81]
Reactive B=12, R=5	N/A	3.03	[2.58, 3.48]	17.85	[16.25, 19.36]

Reactive cells chosen most often



4.4.3

3) The reactive effort gap ((2) - (4)) has mean 0.14, minimum -40 and maximum 45. Reactive effort helps in 17.6% of the scenarios and hinders in 14.9%, not having an effect on most scenarios. The worst 10% have wind coming from the south-west (mean -0.72 radians) and ignition around rows

10-11, columns 9-10. Since this fire will bypass the barrier, reactive placement is targeted, facing south-west, to intercept the fire that the permanent plan misses. The negative gap occurs when the MILP places cells suboptimally, cells which redirect fire towards unprotected weighted areas, which is due to the time limit preventing the finding of an optimal solution.

4) Increasing R from 2 to 8 improves performance overall; R=8 achieving the lowest mean damage (2.1) and $CVaR_{0.9}$(13.8). Beyond R=8, additional reactive cells hinder; R=10 has worse performance, suggesting the reactive MILP cannot find useful placements for extra cells in the time limit. R=4 being worse than R=2 reflects a limitation, where with more cells to place, the search space widens and the solver finds it difficult to return good solutions in the time limit. R=6-8 provides the best balance between a flexible model and reliability of the solver.

4.4.4

1) A fully optimal approach would jointly optimise placing permanent and reactive cells across all scenarios simultaneously rather than fixing permanent cells first and solving each scenario independently, making it more realistic but more expensive.

2) No. (3) commits all 17 cells before the season using no wind Step 2 MILP, whilst (4) reserves 4 cells for after wind is revealed. In theory, (4) should outperform (3) since it uses the same budget with more information. But (3) outperforms (4); 13 permanent cells leaves gaps that 17 would complete, and R=4 reactive cells cannot fully compensate for the gaps left in the permanent barrier for fire to flow through. Confidence intervals from (3) and (4) don't overlap ([1.43, 2.02] vs [2.08, 2.84]), suggesting committing all 17 cells permanently outperforms the reactive split statistically. 17.6% of scenarios where reactive effort helps, confirms more information would matter but only with a better approximation than southernmost ignition and no-wind.

3) The reactive cells chosen will not be the best possible 4 cells for that certain scenario. This is a modelling limitation: under southernmost ignition with no wind, permanent firebreak already blocks all reachability to targets, so the MILP finds barely any additional cells worth adding. This explains why reactive performance is similar to Model B, B=13 and not as good as Model B, B=17. A plan optimised for the wrong scenario (southern ignition with no wind), cannot protect against the actual random ignition and wind conditions, explaining higher damage in practice, and affecting CVaR more than the mean.

4) Reactive cells are along the west side of the landscape, going towards the fixed ignition row. This reveals the permanent plan's limitation of blocking fire from the south but leaving the west unprotected.

5) No, the majority of reactive cells are near the wetland (north-west), which suggests that with westerly wind, the wetland faces more pressure from fire than the settlement, even though the damage to the settlement would be more detrimental. The reactive MILP responds to fire direction and will protect which target faces more pressure.

6) Since wind is revealed before solving the MILP, ignition location and fire spread is left random. If we replace Monte Carlo with a deterministic approach, we would test different wind speeds and wind directions, solving the MILP for each scenario. This gains efficiency and removes statistical

uncertainty. It would lose probabilistic weighting, like how 70% of scenarios have westerly wind, but with deterministic, it would have the same chance occurring as any other possible direction, which is not accurate to reality.

7) A reactive B=12, R=5 plan performs worse than B=13, R=4. Fewer permanent cells weakens baseline protection across all scenarios, and the extra reactive cell cannot compensate. Under no wind conditions, the permanent plan handles everything optimally, leaving no useful additional cells for the reactive solver to add. This means that early, permanent protection is more valuable than reactive protection, and moving budget from permanent to reactive, worsens performance consistently.

Discussion

Step 1 identified where fire concentrates. Step 2 found it cost effective to block early under fixed conditions. Step 3 revealed this plan is fragile under wind. Step 4 confirmed that reactive adjustment cannot fix this cheaply.

Max-flow analysis showed that fire concentrates near the targets columns 14-21 (settlement) and columns 1-8 (wetland), which are far north of the current corridor at row 10. MILP found that under predetermined conditions, blocking fire early near the southernmost ignition front is more budget-efficient than targeting bottleneck corridors. Earlier blocking is cheaper under fixed ignition, no wind, so the MILP places a barrier near the ignition front rather than near targets. Monte Carlo shows the limitation to this plan however. Under wind coming from the south-west, fire is pushed around the horizontal barrier. The B=13 plan still outperforms the current corridor (mean 3.165 vs 3.76 and $CVaR_{0.9}$17.06 vs 24.32) using fewer cells which shows that target placement beats broad coverage, even with probabilistic conditions. This confirms the proposal's basis; row 16 places the barrier where reachability is high enough to block dominant fire paths under the most probable wind conditions.

The reactive analysis shows that adding reactive cells provides limited improvement with the approximations. The reactive MILP returned an optimal objective value 70, identical to the permanent plan's no wind damage, indicating the permanent firebreak already blocks all reachability under southernmost ignition and no wind. Asymmetric damage weights influenced the bottleneck plan but had less effect on MILPs, which place horizontal barriers to protect both targets simultaneously. However, they do surface in the reactive component. Cells near the wetland are chosen more frequently as the MILP responds to which target faces more fire pressure under wind rather than which carries higher damage cost.

Based on the analysis, the authority should replace the current corridor with a barrier of 17 permanent cells along row 16. It achieved the best risk profile (mean 1.73, CVaR 9.76), accepting that no fixed plan fully protects against all wind scenarios. Further work should consider if a horizontal barrier remains optimal under the westerly wind distribution. A diagonal barrier oriented perpendicular to the dominant wind direction (-0.86 radians) may outperform a horizontal one across the scenario distribution. The model assumes ignition is uniformly distributed over vegetated cells; if data suggested clustering near the southern corridors, the permanent plan placement would shift accordingly.

AI Usage Disclaimer

I used AI to debug issues with integrating rAMPL. For writing, it helped proofread and condense my report down to the word limit by addressing spelling, grammar issues and removing and replacing words to make sentences concise. The formulations, AMPL model structure, discussion answers and conclusions are all my own work. I also asked for clarity and understanding as to why (3) is better than (4) in Step 4, and clarify why the reactive MILP returns objective value 70, which helped me understand the modelling limitation. I learned to critically verify AI suggestions which deepened my understanding of the code and the models, as well as learning how to write more concisely. Several recommendations introduced incorrect changes and bugs I had to identify and revert.